



Wrocław
University
of Science
and Technology

Software System Development (Level 2) (Lecture)

The Unified Process

Marek Krótkiewicz

`Marek.Krotkiewicz@pwr.edu.pl`

Department of Applied Informatics
Faculty of Information and Communication Technology
Wrocław University of Science and Technology

June 1, 2026

Agenda

Introduction

Software best practices

- An iterative and incremental process

- Manage the requirements

- Component-based architecture

- Visually model software

- Continuously verify software quality

- Control changes to software

Process structure

- Inception

- Elaboration

- Construction

- Transition

- Development cycle



1

Consultations

Consultations

Consultations: Summer 2026

- Tuesday:

1:00 p.m. – **3:00** p.m. (13:00 – 15:00)

Building: **D2**, Room: **302b**

- Friday:

1:00 p.m. – **3:00** p.m. (13:00 – 15:00)

Zoom platform

The consultations during my office hours are by appointment only. The e-mail with the date and hour of choice should be send at least **2** days in advance with the specific topic of the consultations.

e-mail: Marek.Krotkiewicz@pwr.edu.pl



2

Final test

Final test

Exam: Summer 2026

- I term:

Date: **29 June** 2026

Time: **3:15** p.m. – 3:40 p.m. (15:15 – 15:40)

- II term:

Date: **6 July** 2026

Time: **3:15** p.m. – 3:40 p.m. (15:15 – 15:40)

Rules:

- The exam will take place online.
- It will be possible to join the exam only within the first 5 minutes of its start time.
- To take the exam, you need access to the Internet, a computer and an installed web browser.
- No additional dates are planned.
- The exam result is expressed as a percentage and constitutes an element of the assessment for a group of courses, in accordance with the rules presented below.



3

Introduction



Introduction

The Unified Software Development Process (Unified Process):

- **iterative** and
- **incremental**

software development process framework.

Ivar Jacobson, Grady Booch and James Rumbaugh, The Unified Software Development Process, ISBN 0-201-57169-2, 1999.

Introduction

The Unified Software Development Processes (examples):

- Rational Unified Process (RUP),
- OpenUP,
- Agile Unified Process,
- *others.*

Introduction

Main features of **The Unified Software Development Process**

- an extensible framework,
- Unified Process – generic process.



4

Software best practices

Software best practices

Best practices:

- 1 Develop software iteratively.
- 2 Manage requirements.
- 3 Use component-based architectures.
- 4 Visually model software.
- 5 Continuously verify software quality.
- 6 Control changes to software.



4.1

Software best practices

An iterative and incremental process

Software best practices

An iterative and incremental process

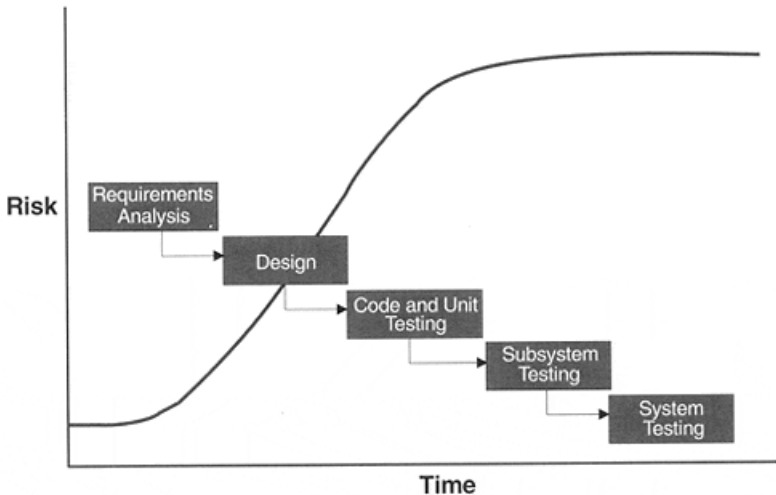


Figure: Traditional approach: waterfall lifecycle

Software best practices

An iterative and incremental process

„If you do not actively attack the risks in your project, they will actively attack you.”

Tom Gilb, *Principle of Software Engineering Management*. Harlow, UK: Addison-Wesley, 1988, p.73.

Software best practices

An iterative and incremental process

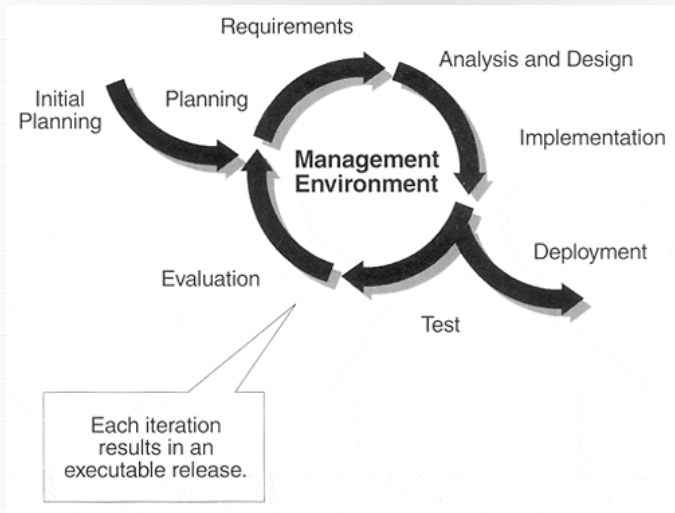


Figure: An **iterative** and **incremental** process

Software best practices

An iterative and incremental process

Developing software **iteratively** – solutions to the root causes:

- 1 serious misunderstandings,
- 2 user feedback,
- 3 focus on critical issues,



Software best practices

An iterative and incremental process

- ④ objective assessment,
- ⑤ inconsistencies,
- ⑥ workload of the team,

Software best practices

An iterative and incremental process

- 7 leverage lessons learned,
- 8 concrete evidence of the project's status.



4.2

Software best practices

Manage the requirements



Software best practices

Manage the requirements

requirement – a condition or capability

The active management of requirements encompasses three activities:

- eliciting, organizing, and documenting the system's required **functionality and constraints**;
- evaluating changes to these requirements and **assessing their impact**;
- tracking and documenting **tradeoffs** and **decisions**.



Software best practices

Manage the requirements

Managing the **requirements** – solutions to the root causes:

- ❶ disciplined approach;
- ❷ communications;
- ❸ prioritized, filtered, and traced requirements;



Software best practices

Manage the requirements

- ④ objective assessment;
- ⑤ inconsistencies;
- ⑥ repository.



4.3

Software best practices

Component-based architecture

Software best practices

Component-based architecture

Important issues:

- perspectives,
- different stakeholders – different agenda to a project,
- system's architecture.

Software best practices

Component-based architecture

A system's architecture encompasses the set of significant **decisions** about:

- organization;
- structural elements;
- behavior;



Software best practices

Component-based architecture

- structural and behavioral elements (composition);
- architectural style.



Software best practices

Component-based architecture

Software architecture is concerned not only with structure and behavior but also with:

- usage,
- functionality,
- performance,
- resilience,
- reuse,
- comprehensibility,
- economic and technologic constraints and trade-offs and
- aesthetic concerns.



Software best practices

Component-based architecture

- reuse, customization;
- COM, CORBA, EJB.



Software best practices

Component-based architecture

Using **component-based architectures** – solutions to the root causes:

- ① resilient architectures,
- ② modularity,
- ③ reuse,



Software best practices

Component-based architecture

- ④ components – configuration management,
- ⑤ visual modeling tools.



4.4

Software best practices

Visually model software



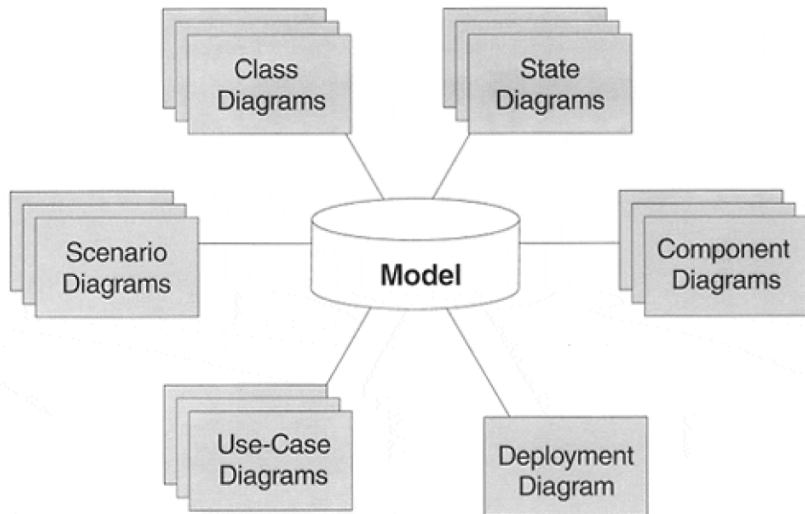
Software best practices

Visually model software

- model – a simplification of reality
- understanding the system
- complex systems – difficult to comprehend such systems in their entirety

Software best practices

Visually model software



Software best practices

Visually model software

Modeling software visually – solutions to the root causes:

- 1 use cases – behavior,
- 2 software design (unambiguously),
- 3 Nonmodularreuse and inflexible,

Software best practices

Visually model software

- ④ details – hidden,
- ⑤ inconsistencies,
- ⑥ application quality – good design,
- ⑦ support for UML.



4.5

Software best practices

Continuously verify software quality

Software best practices

Continuously verify software quality

- software problems after deployment – costs
- quality with respect to:
 - functionality,
 - reliability,
 - performance.

Software best practices

Continuously verify software quality

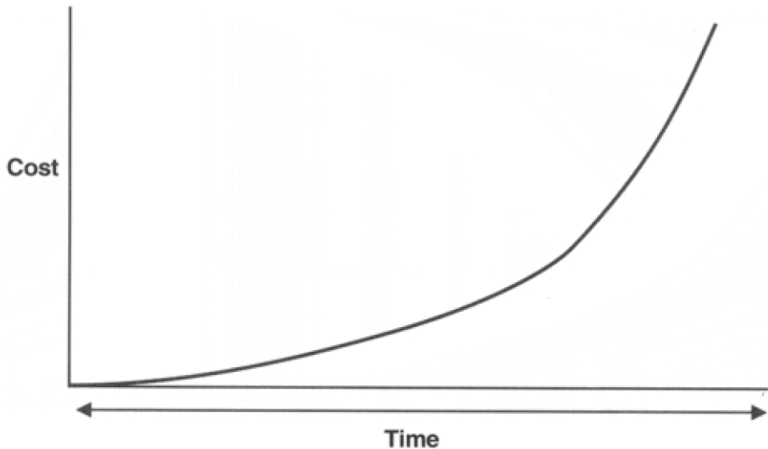


Figure: The cost of fixing problems



Software best practices

Continuously verify software quality

Verifying software quality – solutions to the root causes:

- 1 status assessment is made objective,
- 2 exposes inconsistencies,
- 3 focused on areas of highest risk,



Software best practices

Continuously verify software quality

- ④ defects – identified early,
- ⑤ automated testing,



4.6

Software best practices

Control changes to software

Software best practices

Control changes to software

A key challenge when you're developing software-intensive systems is that you must cope with multiple developers organized into teams, possibly at different sites, working together on multiple:

- iterations,
- releases,
- products, and
- platforms.

Software best practices

Control changes to software

- coordinating the activities
- allocation of resources
- continuously monitor changes – discover and then react to problems



Software best practices

Control changes to software

- coordinating iterations – tested baseline
- maintaining traceability – assessing the impact of change



Software best practices

Control changes to software

Controlling changes – solutions to the root causes:

- ① requirements changes
- ② change requests
- ③ isolated workspaces – reduce interference



Software best practices

Control changes to software

- ④ change rate statistics
- ⑤ workspaces – facilitates consistency,
- ⑥ change propagation
- ⑦ robust, customizable system



5

Process structure

Process structure

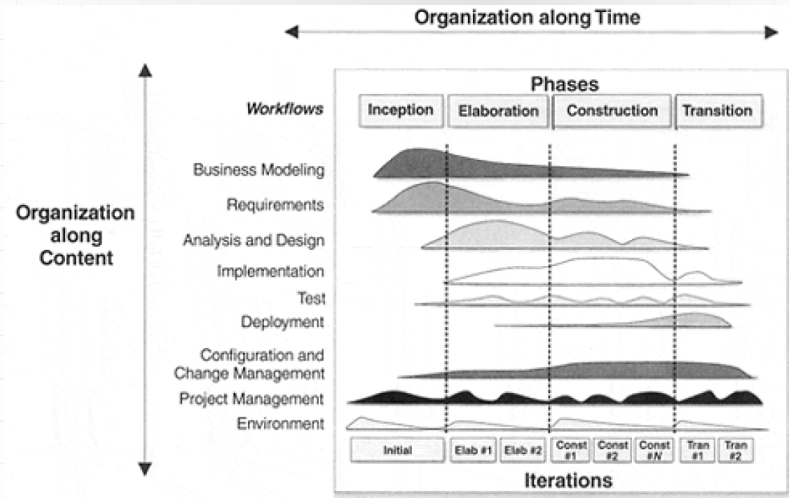


Figure: Process structure

Process structure

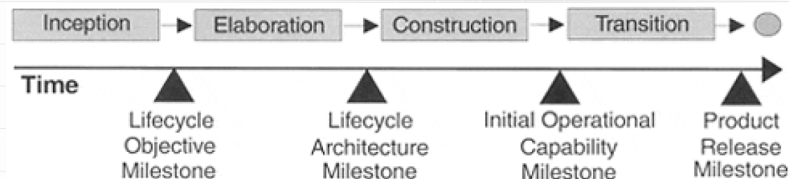


Figure: The four phases and milestones of the iterative process



5.1

Process structure

Inception



Process structure

Inception

Inception – The good idea-specifying the end-product vision and its business case and defining the scope of the project.



Process structure

Inception

The overriding goal of the inception phase is to achieve concurrence among all stakeholders on the lifecycle objectives for the project.

The primary **objectives** of the inception phase include the following:

- scope and boundary conditions
- critical use cases
- at least one candidate architecture
- overall cost and schedule
- risks

Process structure

Inception

The essential **activities** of the inception phase are as follows:

- scope of the project
- business case
- a candidate architecture



Process structure

Inception

The **outcome** of the inception phase is creation of these artifacts:

- vision document
- use-case model
- glossary
- business case
 - context
 - criteria
 - financial
- risk
- plan

Process structure

Inception

The **evaluation criteria** for the inception phase are as follows:

- Stakeholder concurrence on scope definition and cost and schedule estimates
- Requirements understanding as evidenced by the fidelity of the primary use cases
- Credibility of the cost and schedule estimates, priorities, risks, and development process
- Depth and breadth of any architectural prototype that was developed
- Actual expenditures versus planned expenditures



5.2

Process structure

Elaboration



Process structure

Elaboration

Elaboration – Planning the necessary activities and required resources; specifying the features and designing the architecture.



Process structure

Elaboration

The purpose of the elaboration phase is to analyze the problem domain, establish a sound architectural foundation, develop the project plan, and eliminate the project's highest-risk elements.

Process structure

Elaboration

The primary **objectives** of the elaboration phase include the following:

- Define, validate, and baseline the architecture as rapidly as practical.
- Baseline the vision.
- Baseline a high-fidelity plan for the construction phase.
- Demonstrate that the baseline architecture will support this vision for a reasonable cost in a reasonable time.

Process structure

Elaboration

The essential **activities** of the elaboration phase are as follows:

- vision
- process, infrastructure, development environment

Process structure

Elaboration

- plan
- architecture



Process structure

Elaboration

The **outcomes** of the elaboration phase are as follows:

- use-case model
- requirements
- software architecture
- architectural prototype



Process structure

Elaboration

- revised risk list
- development plan
- updated development case
- user manual (optional)

Process structure

Elaboration

The main evaluation criteria for the elaboration phase involve the answers to the following questions:

- Is the vision of the product stable?
- Is the architecture stable?
- Does the executable demonstration show that the major risk elements have been addressed and credibly resolved?
- Is the construction phase plan sufficiently detailed and accurate? Is it backed up with a credible basis for the estimates?
- Do all stakeholders agree that the current vision can be achieved if the current plan is executed to develop the complete system, in the context of the current architecture?
- Is the actual resource expenditure versus planned expenditure acceptable?



5.3

Process structure

Construction



Process structure

Construction

Construction – Building the product and evolving the vision, the architecture, and the plans until the product-the completed vision—is ready for delivery to its user community.



Process structure

Construction

During the construction phase, all remaining components and application features are developed and integrated into the product, and all features are tested thoroughly.



Process structure

Construction

Many projects are large enough that parallel construction increments can be spawned. These parallel activities can significantly accelerate the availability of deployable releases; they can also increase the complexity of resource management and workflow synchronization.



Process structure

Construction

Primary construction phase **objectives** include the following:

- Minimize development costs by optimizing resources and avoiding unnecessary scrap and rework.
- Achieve adequate quality as rapidly as practical.
- Achieve useful versions (alpha, beta, and other test releases) as rapidly as practical.



Process structure

Construction

The essential **activities** of the construction phase are as follows:

- Resource management, resource control, and process optimization
- Complete component development and testing against the defined evaluation criteria
- Assessment of product releases against acceptance criteria for the vision



Process structure

Construction

The **outcome** of the construction phase is a product ready to put in the hands of its end users. At minimum, it consists of the following:

- The software product integrated on the adequate platforms
- The user manuals
- A description of the current release

Process structure

Construction

The **evaluation criteria** for the construction phase involve answering the following questions:

- Is this product release stable and mature enough to be deployed in the user community?
- Are all stakeholders ready for the transition into the user community?
- Are the actual resource expenditures versus planned expenditures still acceptable?



5.4

Process structure

Transition



Process structure

Transition

Transition – Transitioning the product to its users, which includes manufacturing, delivering, training, supporting, and maintaining the product until users are satisfied.



Process structure

Transition

The purpose of the transition phase is to move the software product to the user community.

After the product has been given to the end user, issues usually arise that require you to develop new releases, correct some problems, or the finish features that were postponed.



Process structure

Transition

The transition phase is entered when a baseline is mature enough to be deployed in the end-user domain.

Process structure

Transition

This phase **includes** the following:

- Beta testing to validate the new system against user expectations
- Parallel operation with the legacy system that the project is replacing
- Conversion of operational databases
- Training of users and maintainers
- Rollout of the product to the marketing, distribution, and sales teams



Process structure

Transition

The transition phase concludes when the deployment baseline has achieved the completed vision. For some projects, this lifecycle endpoint may coincide with the starting point of the next cycle, leading to the next generation or version of the product.

For other projects, it may coincide with a delivery of the artifacts to a third party responsible for operation, maintenance, and enhancements of the delivered system.



Process structure

Transition

The transition phase focuses on the activities required to place the software into the hands of the users.

Process structure

Transition

The primary **objectives** of the transition phase include the following:

- Achieve user self-supportability.
- Achieve stakeholder concurrence that deployment baselines are complete and consistent with the evaluation criteria of the vision.
- Achieve final product baseline as rapidly and cost-effectively as practical.



Process structure

Transition

The essential **activities** of the transition phase are as follows:

- Deployment-specific engineering, that is, cutover, commercial packaging and production, sales rollout, and field personnel training
- Tuning activities, including bug fixing and enhancement for performance and usability
- Assessing the deployment baselines against the vision and the acceptance criteria for the product



Process structure

Transition

In the transition phase, the activities performed during an iteration depend on the goal; for fixing bugs, implementation and testing are usually enough.

If new features must be added, the iteration is similar to those of the construction phase.

Process structure

Transition

The primary **evaluation criteria** for the transition phase involve the answers to the following questions:

- Is the user satisfied?
- Are the actual resources expenditures versus planned expenditures still acceptable?



5.5

Process structure

Development cycle

Process structure

Development cycle

- four phases – constitute a development cycle

Unless the life of the product stops at this point, a product will evolve into its next generation by a repetition of the sequence of inception, elaboration, construction, and transition phases, but with different emphases on the various phases – evolution cycles.

Process structure

Development cycle

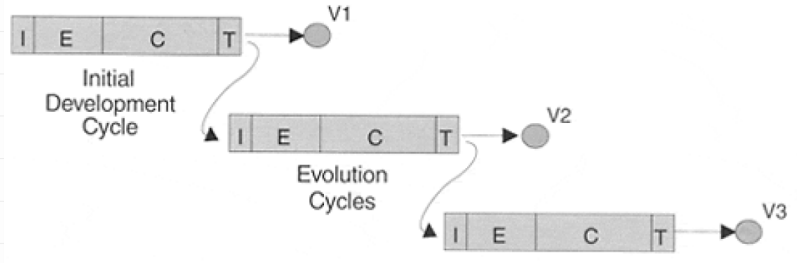


Figure: Initial and evolution cycles

Process structure

Development cycle

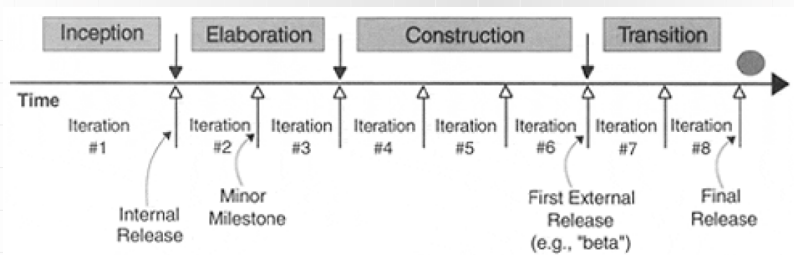


Figure: Iterations

Each iteration follows a pattern similar to the water-fall approach.

Process structure

Development cycle

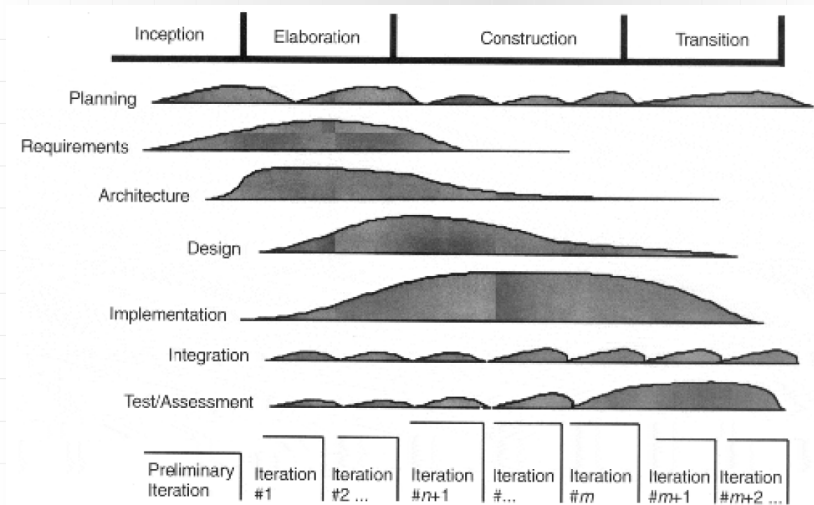
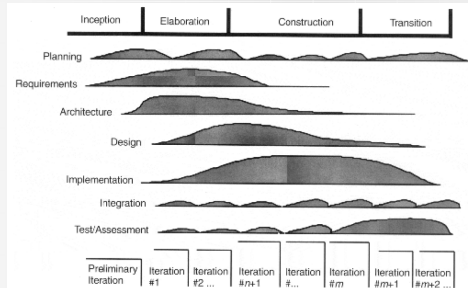


Figure: Importance of activities across one development cycle

Process structure

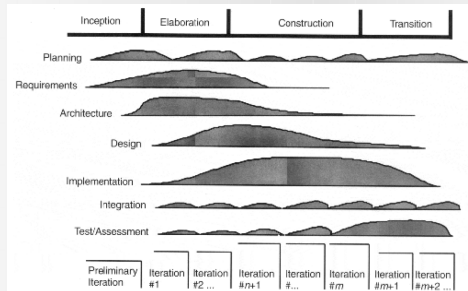
Development cycle



Inception phase – the focus is on understanding the overall requirements and determining the scope of the development effort.

Process structure

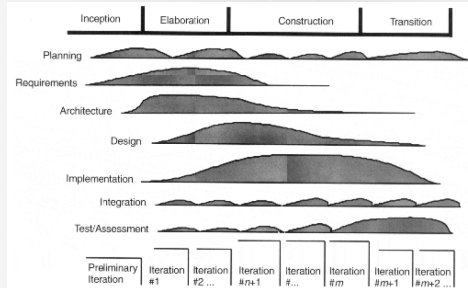
Development cycle



Elaboration phase – the focus is on requirements, but some software design and implementation is aimed at prototyping the architecture, mitigating certain technical risks by trying solutions, and learning how to use certain tools and techniques.

Process structure

Development cycle

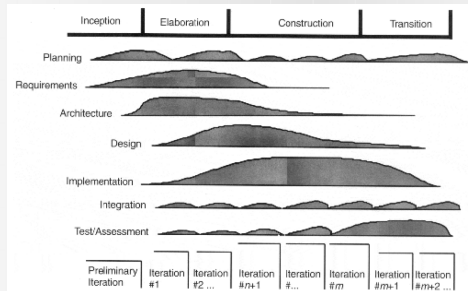


Construction phase – the focus is on design and implementation.

Evolve and flesh out the initial prototype into the first operational product

Process structure

Development cycle



Transition phase – the focus is on ensuring that the system has the right level of quality to meet your objectives; you fix bugs, train users, adjust features, and add missing elements.

Produce and deliver the final product.

Thank you for your attention